bcast/binomial

An AgentMPI collective scaling analysis

Abstract

The binomial tree is AgentMPI's default broadcast for $p > 3$, and this sweep is the evidence for that default. Across 21 measured sizes from $p = 2$ to $p = 32$ it logged exactly $p - 1$ messages — the same traffic as `flat` and `chain` — while its critical path stepped as $\lceil \log_2 p \rceil$: one round at $p = 2$, two at $p = 3$ and 4, three from $p = 5$ to $p = 8$, four from $p = 10$ to $p = 16$, five from $p = 20$ to $p = 32$. Every size agrees with the closed form on both axes and the collective's self-report agrees with the fabric's transport log at all 21, so there is no defect to report. The sweep's most useful finding is a distinction the round count hides: the implementation also records `tree_depth`, the number of hops from the root to each rank, and that quantity equals $\lceil \log_2 p \rceil$ only at powers of two — at all eight non-power-of-two sizes it is exactly one less. The step count and the path length are different things, and the trace measures both. The single most important thing to take away is why a logarithmic broadcast is admissible at all: across all thirteen sizes the thirty-one messages of the $p = 32$ run carried exactly *one* distinct content digest, so a three-, four- or five-deep tree delivers byte-identical content to the last rank. Depth costs latency here and nothing else, and fold depth is zero at 21 of 21 sizes.

1 What this family is

The `bcast` collective under the `binomial` algorithm, measured at 21 process counts from 2 to 32. Every run is agent-free and deterministic, so the measurements are of the protocol itself.

2 What the analysis notices

- **[ok]** All 21 measured sizes logged exactly the number of messages the closed form predicts.
- **[note]** Wall times span 0.005–0.116 s with no agent in the loop, so these measure the fabric and the protocol rather than model latency.

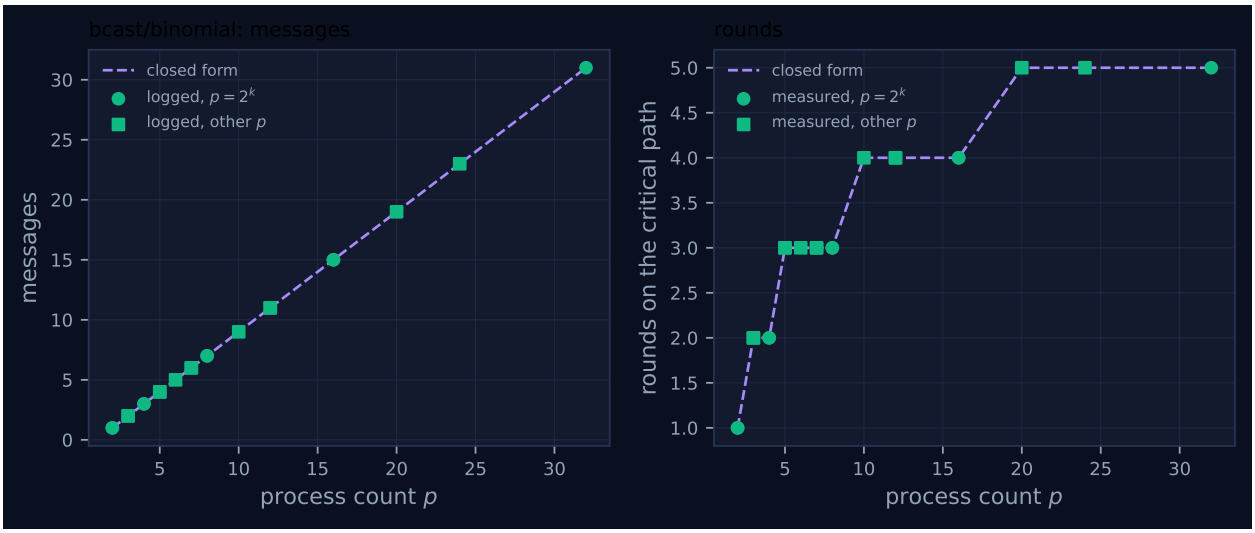

Figure 1: Measured message count and critical-path rounds against process count, with the closed-form prediction. Powers of two are drawn as circles and other sizes as squares, because algorithms take a different code path at sizes that are not powers of two. A red cross marks a size where the collective's own reported count disagrees with the traffic the fabric logged.

3 Every measured size

p	campaign	logged	pred.	self-rep.	rounds	pred.	wall (s)	model	acct.
2	`mb-free`	1	1	1	1	1	0.013	✓	✓
2	`mb-smoke`	1	1	1	1	1	0.005	✓	✓
3	`mb-free`	2	2	2	2	2	0.013	✓	✓
3	`mb-smoke`	2	2	2	2	2	0.012	✓	✓
4	`mb-free`	3	3	3	2	2	0.014	✓	✓
4	`mb-smoke`	3	3	3	2	2	0.014	✓	✓
5	`mb-free`	4	4	4	3	3	0.015	✓	✓
5	`mb-smoke`	4	4	4	3	3	0.014	✓	✓
6	`mb-free`	5	5	5	3	3	0.022	✓	✓
7	`mb-free`	6	6	6	3	3	0.029	✓	✓
7	`mb-smoke`	6	6	6	3	3	0.030	✓	✓
8	`mb-free`	7	7	7	3	3	0.026	✓	✓
8	`mb-smoke`	7	7	7	3	3	0.032	✓	✓
10	`mb-free`	9	9	9	4	4	0.040	✓	✓
12	`mb-free`	11	11	11	4	4	0.038	✓	✓
12	`mb-smoke`	11	11	11	4	4	0.034	✓	✓
16	`mb-free`	15	15	15	4	4	0.035	✓	✓
16	`mb-smoke`	15	15	15	4	4	0.051	✓	✓
20	`mb-free`	19	19	19	5	5	0.079	✓	✓
24	`mb-free`	23	23	23	5	5	0.116	✓	✓
32	`mb-free`	31	31	31	5	5	0.075	✓	✓

4 How the algorithm scales

The implementation is the textbook binomial tree rotated by `root`, so any rank may be the source. Each rank computes a virtual rank $vr = (r - root) \bmod p$ and then scans masks $1, 2, 4, \ldots$ upward. It receives from the peer that clears its lowest set bit — $parent = vr - mask$, rotated back — and

then walks the masks strictly below that one downward, sending to $vr \mid mask$ whenever that virtual rank is less than p. Rank $vr = 0$ has no set bit, so it never receives and sends to $vr \mid mask$ for every mask below $2^{\lceil \log_2 p \rceil}$.

Two consequences follow immediately and both are visible in the trace. First, every virtual rank except 0 has exactly one parent, so the message count is $p - 1$ — the same as `flat` and `chain`, which is the premise of the whole comparison. Second, the masks are visited in a definite order, and the root's sends in `mb-free__coll-bcast-binomial-8` go to ranks 4, then 2, then 1: descending mask, largest subtree first, which is what allows the deepest branch to begin propagating before the root has finished issuing. Rank 4 then forwards to 6 and 5, rank 2 to 3, and rank 6 to 7; the run's seven `msg.send` events reconstruct the tree exactly.

The round count is a step count, not a path length, and this is the subtlety the family view makes visible. The mask loop runs $\lceil \log_2 p \rceil$ times, and the implementation records that number as `rounds`. In the same events it records `tree_depth`, which is popcount(vr), the number of hops the payload actually took to reach that rank. At $p = 8$ the maximum `tree_depth` is 3, at rank $7 = 0\text{b}111$, and it equals the round count. At $p = 7$ the maximum is 2: the largest virtual rank present is $6 = 0\text{b}110$, and no rank has three set bits, so no payload travels three hops — yet the round count is still 3, because the root issues three sends in three successive mask steps and the rank that receives in the third step (rank 3, from rank 2) is therefore third in line. Both figures are correct answers to different questions. Checking the measured maximum `tree_depth` against $\max_{v<p} \text{popcount}(v)$ at all 21 sizes gives agreement at 21 of 21.

Figure 1 is these two facts side by side. The left panel is the straight line $p - 1$ with every marker on it, identical to `flat`'s and `chain`'s left panel: the tree buys nothing in volume. The right panel is the staircase, and its treads are at 1 ($p = 2$), 2 ($p = 3, 4$), 3 ($p = 5, 6, 7, 8$), 4 ($p = 10, 12, 16$) and 5 ($p = 20, 24, 32$). The risers sit at $p = 2^k + 1$, because that is where $\lceil \log_2 p \rceil$ increments, and the sweep brackets rather than pins them: it contains 8 and 10 but not 9, and 16 and 20 but not 17. Every measured point falls on the correct tread, and the two points immediately either side of each riser are measured, which is the strongest statement the available sizes support.

The viewer screenshot for `mb-free__coll-bcast-binomial-16` shows the staged propagation the tree implies. The collectives panel reports 16 invocations of `bcast`, algorithm `binomial`, maximum rounds 4, 15 messages, maximum fold depth 0; the stats row reports 0 agent calls, 0.0k tokens and $0.000. In the timeline, rank 0 carries a tight cluster of three or four message glyphs early, ranks 8, 4, 2 and 1 carry their own smaller clusters shortly after, and the leaves carry single ticks last. There is no lane that sits idle through the whole span and no lane that carries a long bar — the run is 15 instantaneous message events across 16 lanes in 0.04 s, and the only structure is the fan-out cascade.

5 Powers of two and everything else

The non-power-of-two path in this algorithm is a single guard, `if child_v < p`, which suppresses sends to virtual ranks that do not exist. That guard is the entire remainder handling, and it behaves. All eight non-power-of-two sizes in the sweep logged exactly $p - 1$ messages (2 at $p = 3$, 4 at $p = 5$, 5 at $p = 6$, 6 at $p = 7$, 9 at $p = 10$, 11 at $p = 12$, 19 at $p = 20$, 23 at $p = 24$), all eight reported the same count the fabric logged, and all eight recorded the round count $\lceil \log_2 p \rceil$. There are no red crosses in Figure 1.

The interesting non-power-of-two behaviour is the one already described, and it is a property of the tree rather than a defect. At every power of two the maximum `tree_depth` equals the round count;

at every one of the eight other sizes it is exactly one less — 1 against 2 at $p = 3$, 2 against 3 at $p = 5, 6, 7$, 3 against 4 at $p = 10, 12$, 4 against 5 at $p = 20, 24$. So the tree at a non-power-of-two size is slightly *shallower* than $\lceil \log_2 p \rceil$ and its critical path is nonetheless $\lceil \log_2 p \rceil$ steps, because the root's fan-out is $\lceil \log_2 p \rceil$ sends issued in sequence. A harness that read `tree_depth` as a latency figure would under-predict at those sizes; a harness that read `rounds` as a hop count would over-predict. The implementation exposing both is the reason a reader can tell.

Token accounting is exact at all 21 sizes: the sum of the `tokens` field over `msg.send` events is $p - 1$ and the collective reports $p - 1$ as `tokens_sent`. The binomial broadcast forwards `result` — the handle it received — with no metadata wrapper, so there is nothing for the two views of the volume to disagree about. This is not true of `reduce/binomial`, which wraps its accumulator in a four-field envelope and under-reports the difference.

Eight sizes were measured twice, under `mb-free` and `mb-smoke`. At all eight, every count is identical between campaigns; the run wall times differ by factors from 1.01 to 2.92, the largest at $p = 2$ where the absolute times are 13.5 and 4.6 ms. Two independent measurements agreeing on every integer and disagreeing on every duration is a statement about which of the two the sweep is competent to measure.

6 When to choose this algorithm

For broadcast, essentially always, and the reason is the digest evidence rather than the round count. All three broadcast algorithms send $p - 1$ messages, so they differ only in depth, and the question is whether depth costs anything besides time. For a broadcast in AgentMPI it does not, and the sweep shows it: across all thirteen sizes and all three algorithms, the set of distinct digests carried by the collective's `msg.send` events has size exactly one. At $p = 32$ under `binomial`, thirty-one messages traversing a five-step, five-deep tree carry the single digest the root put in, and `fold_depth` is 0 in all 32 `coll.bcast` records. Interior ranks forward a content-addressed handle, so a five-deep tree is not a game of telephone — it is five hops of pointer copying. The contrast is stark against the mirrored reduction: `reduce/binomial` and `reduce/chain` carry $p - 1$ *distinct* digests at every size, one new artifact per message, even though every rank contributed the identical value 1 under `SUM`. That asymmetry is what makes depth a pure latency cost for a broadcast and a potential fidelity cost for a reduction.

Against `chain` the case is decisive and the sweep quantifies it. Both send $p - 1$ messages; chain's critical path is $p - 1$ hops and binomial's is $\lceil \log_2 p \rceil$ steps, a predicted ratio of $31/5 = 6.2$ at $p = 32$. Measured maximum per-rank blocking is 44.8 ms for binomial against 1,481.8 ms for chain, a factor of 33. The measured advantage is five times larger than the model's, and the excess is chain's per-hop cost growing with p (from 10.4 ms per hop at $p = 2$ to 47.8 ms at $p = 32$, as thirty-two threads contend on one SQLite file) rather than anything binomial does well. The model's 6.2 is the figure to quote; the measured 33 is the figure that happens to hold on this harness.

Against `flat` the case is weaker than it looks, and honesty requires saying so. Flat's network depth is one — every leaf receives directly from the root — and its $p - 1$ "rounds" are the root's send loop, which measured 0.90 ms per send at $p = 32$. Flat's maximum per-rank blocking at $p = 32$ is 28.0 ms against binomial's 44.8 ms: flat is *faster* here. The model disagrees (3.97 s against 0.64 s at $n = 1000$ tokens) because it charges one prompt-processing term per round, and this sweep admitted zero tokens into any context across 1,110 `msg.recv` events, so the term that dominates the prediction is the term the measurement zeroes. The defensible reasons to prefer binomial over flat are therefore

structural rather than measured: it removes the root as a serialisation point, it spreads fan-out across $\lceil \log_2 p \rceil$ levels so that no single rank must issue $p - 1$ sends before returning, and it staggers context admission if the harness admits. At $p \leq 3$ the two are identical and `bcast` defaults to `flat`, which the implementation does explicitly.

7 Threats to this reading

The wall times at large p do not measure the collective. In `mb-free__coll-bcast-binomial-32` the thirty-two `rank.init` events span 7.3 to 63.0 ms and the entire run lasts 75.0 ms, so most of the run is ranks starting up. The 0.075 s in the generated table and the 49.8 ms spread between the first and last rank to record the collective are both dominated by launch stagger. The only duration in this family I would defend is the maximum per-rank blocking inside the collective, and at $p = 32$ that is 44.8 ms of SQLite writes and event appends.

These runs contain no agents, and the consequence is sharper for a tree than for a chain. Every one of the 21 runs reports `executors = {"none": p}`, zero `agent.call` events, zero busy time, an idle fraction of 1.0 and \$0.000. The argument for logarithmic broadcast rests on α being large, and α is exactly zero here. What the sweep establishes is that the implementation's structure is the structure the closed form describes; that the structure is worth having is an inference from the cost model, and the model's coefficients are untested by these runs. Relatedly, the model charges a broadcast no α at all, on the grounds that forwarding a handle invokes no agent. The sweep is consistent with that premise but cannot confirm it, because a harness that chose to admit the payload into a forwarding rank's context would incur a cost this sweep never pays: zero of 1,110 receptions admitted a single token.

Several code paths are unexercised. All 21 runs used `root = 0`, so the rotation $vr = (r - root) \bmod p$ that makes any rank a valid source is never driven with a non-zero root — and that rotation is the part of the binomial implementation most likely to be subtly wrong, because it is what forces `reduce/binomial` to refuse non-commutative operators at non-zero roots. All 21 used a one-token scalar payload, so the volume term $(p - 1)n$ is validated only at $n = 1$. And the sweep contains no $p = 9$ or $p = 17$, so the two round-count risers between $p = 8$ and $p = 32$ are bracketed by measured points but not located by them.

Finally, the `tree_depth` claim is a comparison between one number the implementation reports and one I computed from the definition of the tree, not between two independent measurements. It would be stronger if the depth were recovered from the transport log by tracing each message's source back to the root; the digests cannot do that here, because all $p - 1$ messages carry the same digest, which is the very property the previous section relies on.